

DuerLink 配网 SDK 接入文档

版本：V 1.1

度秘事业部

修改纪录

版本	修改日期	修改人
1.0	2017/12/27	王春虎，李鹏，于耀辉
1.1	2018/01/22	修改配网初始 化参数王春虎

目录

一、	简介	4
二、	基础依赖条件	4
三、	工作流程	4
四、	源文件说明	6
五、	接入流程	6
六、	完整示例代码	14
七、	配网配对注意事项	14

一、简介

DuerLink 提供无线网络配置、配对（设备发现与账户绑定）、DLP 等基础功能。其中无线配网功能目前支持 SoftAp，蓝牙两种方式。因为配网过程中，涉及无线网卡模式切换、AP 热点启动等与系统强相关操作，Duerlink 需要用户在 so 库基础上进行二次开发，才可以在用户设备上使用配网功能。

二、基础依赖条件

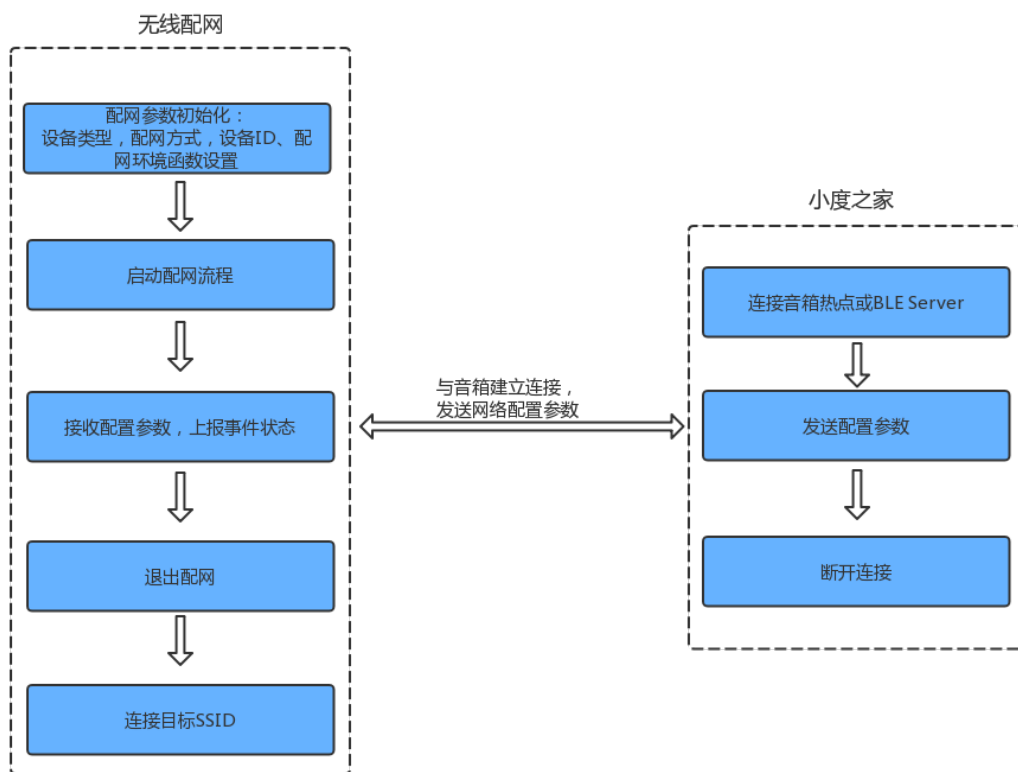
1. 无线配网
- 若采用 SoftAp 方式，需要设备无线网卡支持 AP 模式；
 - 若采用蓝牙方式，需要支持 BLE。
 - 配网时，设备 AP 热点名称，或者建立 BLE Server 需要命名为 DuerOS_XXXX 格式。例如：设备 MAC 地址为 c4:b3:01:ce:e0:51 ，热点名称可命名为 DuerOS_E051。
 - BLE 相关 UUID 设置如下：

SERVICE	00001111-0000-1000-8000-00805f9b34fb
CHARACTERISTIC	00002222-0000-1000-8000-00805f9b34fb
DESCRIPTOR	00002902-0000-1000-8000-00805f9b34fb

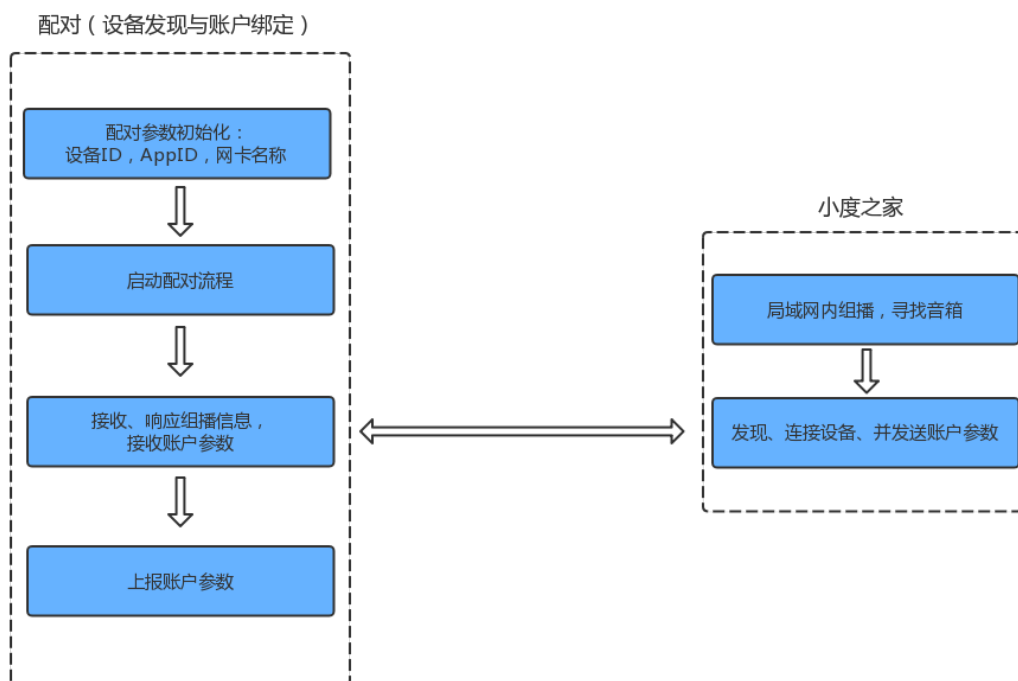
2. 配对
- 在设备发现与账户绑定阶段，音箱会进行组播操作。需要所在局域网路由器允许组播。

三、工作流程

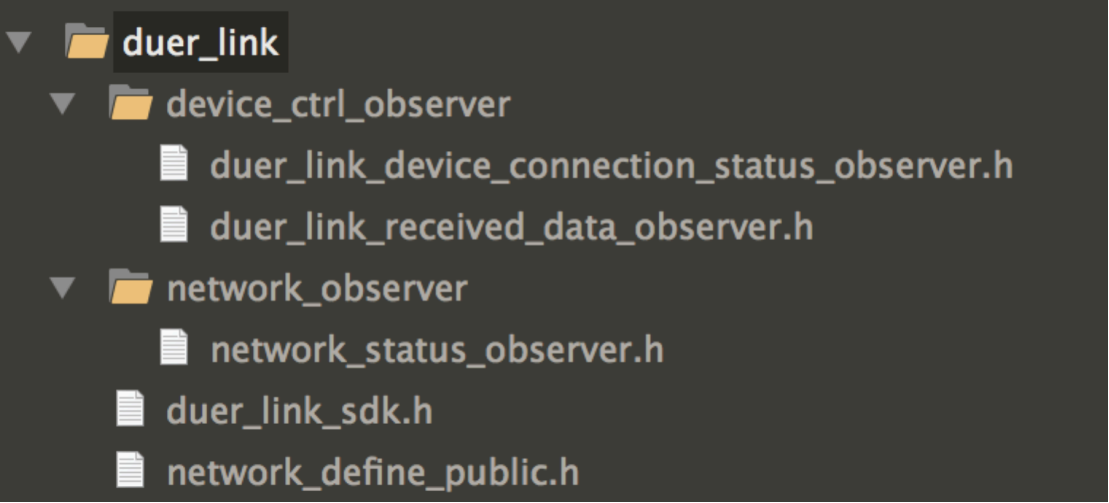
1. 无线配网



2. 配对



四、源文件说明



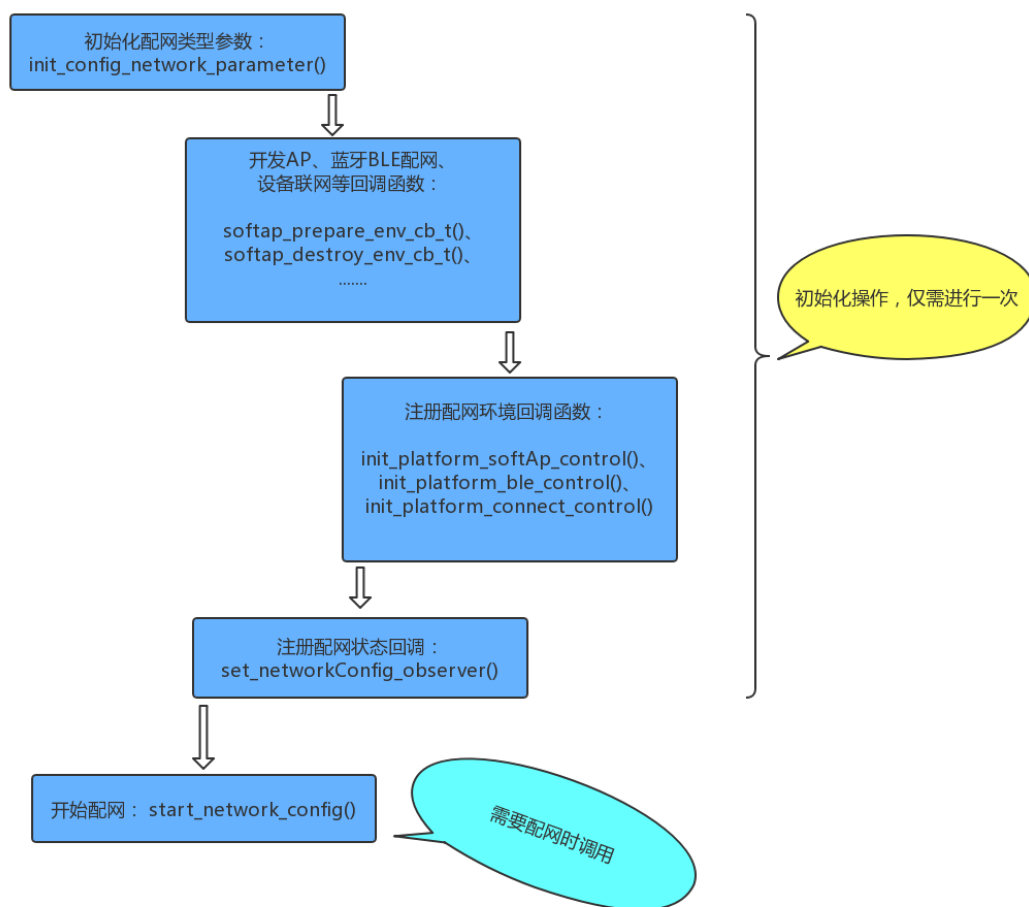
DuerLink 源文件目录

duer_link_sdk.h	配网、配对操作等 API
network_define_public.h	设备类型，操作类型等定义
network_status_observer.h	配网事件状态回调
duer_link_received_data_observer.h	DLP 接受数据 observer
duer_link_device_connection_status_observer.h	音箱控制场景设备状态 observer

五、接入流程

* API、操作类型定义等详细说明请参照 duer_link 头文件。

- 无线配网
 1. 配网接入流程图



* 参数初始化，配网环境回调函数注册等，仅需要一次调用。配网操作启动函数 `start_network_config()` 需要时调用。

2. 初始化配置参数

```
/**
 * 初始化配网参数
 * @param speaker_type 设备类型
 * @param autoType 配网方式
 * @param devicedID 设备ID
 * @param ap_interface AP网卡名称
 */
void init_config_network_parameter(platform_type speaker_type,
                                  auto_config_network_type autoType,
                                  std::string devicedID,
                                  std::string ap_interface);
```

调用示例：

设置设备类型为 `EOther`，配网方式为 `AP` 模式。

```
duerLinkSdk *instance = duerLinkSdk::get_instance();
instance->init_config_network_parameter(duerLink::platform_type::E0ther,
                                         duerLink::auto_config_network_type::ESoftAp,
                                         "DeviceId");
```

3. 设置配网环境操作函数

配网环境相关操作、以及设备联网操作与系统强相关，需要用户进行二次开发，并以回调的形式在 DuerLink 中注册。

1) SoftAP 配网

编写 SoftAP 配网环境准备函数：

```
bool softap_prepare_env_cb() {
    //停止无线网络连接
    stop_wpa_suppliment();
    //切换无线网卡模式
    echo_ap_to_devwifi();
    // 启动AP网卡
    starup_ap_interface();
    //设置AP网卡IP地址为192.168.1.1
    set_ap_tmp_ipaddr();
    //启动AP热点
    start_ap_mode();
    //启动dhcp服务。
    start_dhcp_server();
    return true;
}
```

编写 SoftAP 配网环境销毁函数：

```
bool softap_destroy_env_cb() {
    //停止DHCP服务
    stop_dhcp_server();
    //删除AP网卡IP地址。
    delete_tmp_addr();
    //停止AP热点。
    stop_ap_mode();
    //关闭ap网卡
    down_ap_interface();
    return true;
}
```

注册 AP 配网环境函数：

```
instance->init_platform_softAp_control(softap_prepare_env_cb,  
                                         softap_destroy_env_cb);
```

2) 蓝牙配网

编写蓝牙配网环境准备函数：

```
bool ble_prepare_env_cb() {  
    //开启BLE服务  
    framework::DeviceIo::getInstance()->controlBt(framework::BtControl::BLE_OPEN_SERVER);  
    return true;  
}
```

编写蓝牙配网环境销毁函数：

```
bool ble_destroy_env_cb() {  
    //关闭BLE服务  
    framework::DeviceIo::getInstance()->controlBt(framework::BtControl::BLE_CLOSE_SERVER);  
    return true;  
}
```

编写蓝牙数据发送函数：

```
int ble_send_data_cb(char *data, unsigned short data_length) {  
    //发送BLE数据  
    framework::DeviceIo::getInstance()->controlBt(framework::BtControl::BLE_SERVER_SEND,  
                                                    data,  
                                                    data_length);  
    return 0;  
}
```

蓝牙接收数据，请将数据传入 ble_rcv_data 函数

```
bool DuerLinkMtkInstance::ble_rcv_data(void *data, int len) {  
    //BLE接收参数  
    return duerLinkSdk::get_instance()->ble_rcv_data(data, len);  
}
```

注册蓝牙配网环境函数：

```
instance->init_platform_ble_control(ble_prepare_env_cb,  
                                     ble_destroy_env_cb,  
                                     ble_send_data_cb);
```

3) 设备联网函数

编写联网参数：

```
bool platform_connect_wifi_cb(const char *ssid,  
                              const int ssid_len,  
                              const char *pwd,  
                              const int pwd_len) {  
  
    //保存配网参数SSID，密码  
    tmp_wpa_conf(ssid, pwd, DUERLINK_WPA_CONFIG_FILE_MTK);  
  
    //设备连接目标SSID  
    system_command("/usr/bin/connect_ap.sh &");  
  
    return true;  
}
```

注册联网回调：

```
instance->init_platform_connect_control(platform_connect_wifi_cb);
```

4. 注册配网状态回调

```
void DuerLinkMtkInstance::set_networkConfig_observer(NetworkConfigStatusObserver* config_listener) {  
    duerLinkSdk::get_instance()->set_networkConfig_observer(config_listener);  
}
```

5. 开始配网

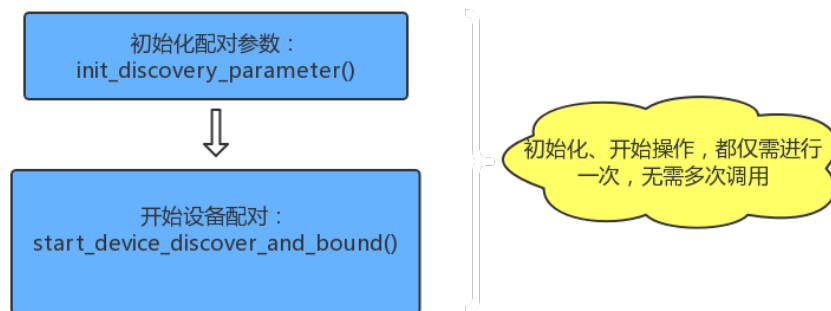
开始配网：

```
duerLinkSdk::get_instance()->start_network_config();
```

* 配网开始之后，小度之家配网参数传输完毕，duerlink 会在内部停止配网工作，不需要额外停止操作。如果想主动终止，请调用 `stop_network_config()` 结束操作。

- 配对

1. 接入流程图



* 配对启动 `start_device_discover_and_bound()` 操作仅需在设备启动时，调用一次，无需多次调用。

2. 初始化配置参数

```
/**
 * 初始化设备参数
 * @param devicedID 设备ID
 * @param appId APPID
 * @param interface STA网卡名称
 * @param bdussFile BDUSS存储路径
 */
void init_discovery_parameter(std::string devicedID,
                             std::string appId,
                             std::string interface,
                             std::string bdussFile);
```

调用示例：

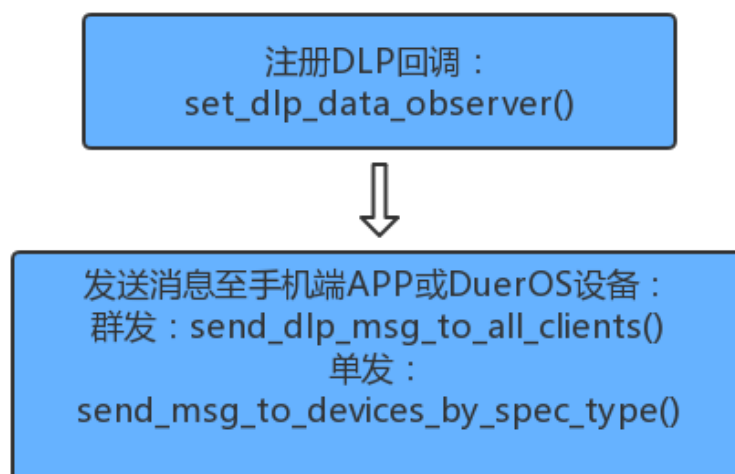
```
duerLinkSdk::get_instance()->init_discovery_parameter(devicedID,
                                                         appId,
                                                         interface,
                                                         "/data/duer/bduss.txt");
```

3. 开始配对

```
duerLinkSdk::get_instance()->start_device_discover_and_bound();
```

● DLP(APP 或 DuerOS 设备控制音箱)

1. 接入流程图



2. 注册 DLP 回调函数

```
/**
void set_dlp_data_observer(DuerLinkReceivedDataObserver* observer);
```

接口说明：当 duer link 收到 APP 或者任何 dueros 设备发送来的消息后，通过注册的回调函数，转发 DLP 消息给外部注册方。

3. 设备发送消息给 APP

```
bool send_dlp_msg_to_all_clients(const std::string& sendBuffer);
```

接口说明：设备端 dlp server 支持多个合法的 APP 建立连接，但是必须是

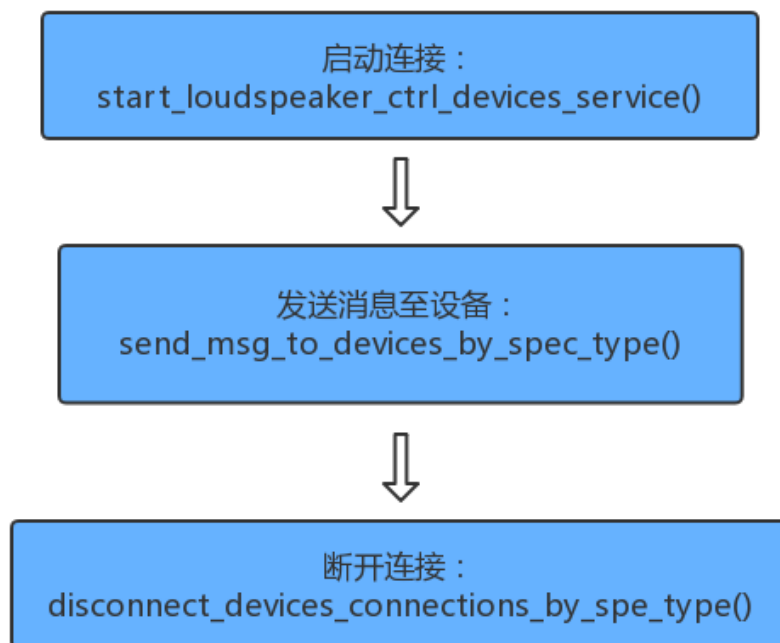
统一账号，此接口支持发送消息给所有合法的连接。

```
bool send_msg_to_devices_by_spec_type(const std::string& msg_buf, device_type device_type_value);
```

接口说明：发送消息给指定的id, id 参数和接受数据回调的函数id保持一致。

- 音箱控制其它 dueros 设备

1. 接入流程图



2. 启动连接设备

```
bool start_loudspeaker_ctrl_devices_service(device_type device_type_value,  
                                            const std::string& client_id = "",  
                                            const std::string& link_buffer = "");  
/**
```

接口说明：启动连接设备。

参数： device_type: 设备类型定义

client_id:通过百度申请的 client id

link_buffer:把云端返回的连接电视 dcs 转为 dlp

3. 发送消息至设备

```
bool send_msg_to_devices_by_spec_type(const std::string& msg_buf, device_type device_type_value);
```

接口说明：发送消息给设备。

参数：msg_buf：云端下发的 dcs 指令转为 dlp 后

device_type：设备类型定义

4. 断开连接的设备

```
bool disconnect_devices_connections_by_spe_type(device_type device_type_value, const std::string& unlink_buffer = "");
```

接口说明：断开连接的设备：

参数：device_type：设备类型定义

unlink_buffer:云端下发的 dcs 断开电视指令 转为 dlp 协议

5. 获取已连接设备信息

```
bool get_curret_connect_device_info(std::string& client_id,  
                                     std::string& device_id,  
                                     device_type device_type_value);
```

接口说明：取得当前连接设备的 client id 和 device id

参数说明：client_id：从百度申请的 client id

device_id：设备 device id

device_type_value：设备类型定义

六、完整示例代码

[duer_link_example.txt](#)

七、配网配对注意事项

a) start_network_config() 为非阻塞操作。

b) AP 配网无线热点和 wifi 的 interface 名字要不一样，建议无线热点接口名字

叫 ap0，wifi 接口名字叫 wlan0。

c) 蓝牙配网请确认音箱设置的 ble name 和 ble uuid 与小度之家设置的过滤规则是一样的。

d) 配网过程中有几个关键节点可以 check 是否成功：

Wifi 二层连接是否成功：`wpa_cli -iwlan0 -p/tmp/wpa_supplicant status` 查看 wifi 连接是否为 connected。/tmp/wpa_supplicant 这个路径在不同的平台可能不一样，需要和 wpa_supplicant.conf 中 ctrl_interface 后面设定的一致。

如果一直连接不成功，请确认 wifi 密码和 wpa_supplicant.conf 内容格式是否正确，如果都正确请用手机使用相同密码连接路由器测试一下。如果手机能连接成功但音箱连接不成功，请找 wifi 驱动工程师定位一下 wifi 驱动问题；如果手机也不能连接成功，请重启路由器重试。

Wifi 三层连接是否成功：通过 `ifconfig` 查看是否获取到正常的 ip 地址，169 开头的 ip 地址不是可用的 ip 地址。

如果一直获取不到 ip，首先请确认 dhcpcd(或类似)进程是否存在，如果 dhcpcd 存在，请用手机或电脑测试能否获取到 ip，最好重启路由器再试一下。如果手机能获取到 ip，但音箱不能获取，请检查一下路由器是否有特殊设置。

小度之家发现音箱设备：如果小度之家显示局域网发现设备失败，请检查路由器是否关闭了组播功能。

另外配网配对过程中可能出现的错误现象，例如 ap interface 和 wifi interface 同时 up，且都有 ip 地址，此时要看 interface 切换过程中某个 interface 是否没有正常 down。

蓝牙配网请确认每次蓝牙收发数据都正常发出去和收到，有些平台在调发送数据的接口后立即关闭蓝牙会导致关闭蓝牙前的蓝牙数据发送失败。